

COSC 231 Lab6: Deadlocks

Task1:

Code:

```

#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_a(void *arg) {
    pthread_mutex_lock(&mutex1);
    printf("Thread A: Locked mutex1\n");
    fflush(stdout);

    usleep(100000);

    printf("Thread A: Waiting for mutex2...\n");
    fflush(stdout);
    pthread_mutex_lock(&mutex2);

    printf("Thread A: Locked mutex2 - Completed!\n");
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

void *thread_b(void *arg) {
    pthread_mutex_lock(&mutex2);
    printf("Thread B: Locked mutex2\n");
    fflush(stdout);

    usleep(100000);

    printf("Thread B: Waiting for mutex1...\n");
    fflush(stdout);
    pthread_mutex_lock(&mutex1);

    printf("Thread B: Locked mutex1 - Completed!\n");
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
    return NULL;
}

int main() {
    pthread_t tA, tB;

    pthread_create(&tA, NULL, thread_a, NULL);
    pthread_create(&tB, NULL, thread_b, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    printf("Both threads completed (this won't print - deadlock!)\n");
    return 0;
}
~

```

Output:

```
dcuev@iFHOX:~/cosc315/lab6$ ./deadlock_demo
Thread A: Locked mutex1
Thread B: Locked mutex2
Thread A: Waiting for mutex2...
Thread B: Waiting for mutex1...
|
```

It hangs

Task 2: Fixing it

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_a(void *arg) {
    // Always acquire mutex1 first, then mutex2
    pthread_mutex_lock(&mutex1);
    printf("Thread A: Locked mutex1\n");
    fflush(stdout);

    usleep(100000);

    pthread_mutex_lock(&mutex2);
    printf("Thread A: Locked mutex2\n");
    fflush(stdout);

    printf("Thread A: Completed!\n");
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

void *thread_b(void *arg) {
    // Same order: mutex1 first, then mutex2
    pthread_mutex_lock(&mutex1);
    printf("Thread B: Locked mutex1\n");
    fflush(stdout);

    usleep(100000);

    pthread_mutex_lock(&mutex2);
    printf("Thread B: Locked mutex2\n");
    fflush(stdout);

    printf("Thread B: Completed!\n");
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

int main() {
    pthread_t tA, tB;

    pthread_create(&tA, NULL, thread_a, NULL);
    pthread_create(&tB, NULL, thread_b, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    printf("Both threads completed successfully!\n");
    return 0;
}

```

Output:

```
dcuev@iFHOX:~/cosc315/lab6$ ./deadlock_fixed
Thread A: Locked mutex1
Thread A: Locked mutex2
Thread A: Completed!
Thread B: Locked mutex1
Thread B: Locked mutex2
Thread B: Completed!
Both threads completed successfully!
dcuev@iFHOX:~/cosc315/lab6$ |
```

Task 3:

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define NUM_PHILOSOPHERS 5
#define NUM_ITERATIONS 3

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    for (int i = 0; i < NUM_ITERATIONS; i++) {

        printf("Philosopher %d: Thinking\n", id);
        fflush(stdout);
        usleep((rand() % 100 + 50) * 1000);

        pthread_mutex_lock(&chopsticks[left]);
        printf("Philosopher %d: Picked up LEFT chopstick %d\n", id, left);
        fflush(stdout);

        usleep(50000);

        printf("Philosopher %d: Waiting for RIGHT chopstick %d...\n", id, right);
        fflush(stdout);
        pthread_mutex_lock(&chopsticks[right]);
        printf("Philosopher %d: Picked up RIGHT chopstick %d - EATING\n", id, right);
        fflush(stdout);

        usleep((rand() % 100 + 50) * 1000);

        pthread_mutex_unlock(&chopsticks[right]);
        pthread_mutex_unlock(&chopsticks[left]);
        printf("Philosopher %d: Put down chopsticks\n", id);
        fflush(stdout);
    }

    printf("Philosopher %d: Done!\n", id);
    return NULL;
}

int main() {
    srand(time(NULL));
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_init(&chopsticks[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_join(threads[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_destroy(&chopsticks[i]);

    printf("All philosophers finished!\n");
    return 0;
}

```

Output:

```
dcuev@iFH0X:~/cosc315/lab6$ ./philosophers_deadlock
Philosopher 0: Thinking
Philosopher 2: Thinking
Philosopher 3: Thinking
Philosopher 4: Thinking
Philosopher 1: Thinking
Philosopher 4: Picked up LEFT chopstick 4
Philosopher 2: Picked up LEFT chopstick 2
Philosopher 0: Picked up LEFT chopstick 0
Philosopher 1: Picked up LEFT chopstick 1
Philosopher 4: Waiting for RIGHT chopstick 0...
Philosopher 2: Waiting for RIGHT chopstick 3...
Philosopher 3: Picked up LEFT chopstick 3
Philosopher 0: Waiting for RIGHT chopstick 1...
Philosopher 1: Waiting for RIGHT chopstick 2...
Philosopher 3: Waiting for RIGHT chopstick 4...
|
```

Task 4:

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define NUM_PHILOSOPHERS 5
#define NUM_ITERATIONS 10

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    int first = (left < right) ? left : right;
    int second = (left < right) ? right : left;

    for (int i = 0; i < NUM_ITERATIONS; i++) {

        printf("Philosopher %d: Thinking (iteration %d)\n", id, i + 1);
        fflush(stdout);
        usleep((rand() % 100 + 50) * 1000);

        pthread_mutex_lock(&chopsticks[first]);
        printf("Philosopher %d: Picked up chopstick %d (first)\n", id, first);
        fflush(stdout);

        pthread_mutex_lock(&chopsticks[second]);
        printf("Philosopher %d: Picked up chopstick %d (second) - EATING\n", id, second);
        fflush(stdout);

        usleep((rand() % 100 + 50) * 1000);

        pthread_mutex_unlock(&chopsticks[second]);
        pthread_mutex_unlock(&chopsticks[first]);
        printf("Philosopher %d: Finished eating (iteration %d)\n", id, i + 1);
        fflush(stdout);
    }

    printf("Philosopher %d: All done!\n", id);
    return NULL;
}

int main() {
    srand(time(NULL));
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_init(&chopsticks[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_join(threads[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_destroy(&chopsticks[i]);

    printf("All philosophers finished without deadlock!\n");
    return 0;
}

```


Output:
5 iterations

[illegible]

10 iterations:

```

$ ./philosophers_fixed
Philosopher 0: Thinking (iteration 1)
Philosopher 1: Thinking (iteration 1)
Philosopher 2: Thinking (iteration 1)
Philosopher 3: Thinking (iteration 1)
Philosopher 4: Thinking (iteration 1)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 4: Finished eating (iteration 1)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 4: Thinking (iteration 2)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 3: Finished eating (iteration 1)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 3: Thinking (iteration 2)
Philosopher 2: Finished eating (iteration 1)
Philosopher 2: Thinking (iteration 2)
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 1: Finished eating (iteration 1)
Philosopher 1: Thinking (iteration 2)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 3: Finished eating (iteration 2)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 3: Thinking (iteration 3)
Philosopher 2: Finished eating (iteration 2)
Philosopher 2: Thinking (iteration 3)
Philosopher 0: Finished eating (iteration 1)
Philosopher 0: Thinking (iteration 2)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 1: Finished eating (iteration 2)
Philosopher 1: Thinking (iteration 3)
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 4: Finished eating (iteration 2)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 4: Thinking (iteration 3)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 0: Finished eating (iteration 2)
Philosopher 0: Thinking (iteration 3)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 3: Finished eating (iteration 3)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 3: Thinking (iteration 4)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 4: Finished eating (iteration 3)
Philosopher 4: Thinking (iteration 4)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 2: Finished eating (iteration 3)
Philosopher 2: Thinking (iteration 4)
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 3: Finished eating (iteration 4)
Philosopher 3: Thinking (iteration 5)
Philosopher 1: Finished eating (iteration 3)
Philosopher 1: Thinking (iteration 4)
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 2: Finished eating (iteration 4)
Philosopher 2: Thinking (iteration 5)
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 0: Finished eating (iteration 3)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 0: Thinking (iteration 4)
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 3: Finished eating (iteration 5)
Philosopher 3: Thinking (iteration 6)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 1: Finished eating (iteration 4)
Philosopher 1: Thinking (iteration 5)
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 4: Finished eating (iteration 5)
Philosopher 4: Thinking (iteration 6)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 0: Picked up chopstick 1 (second) - EATING

```

```

Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 2: Finished eating (iteration 6)
Philosopher 2: Thinking (iteration 7)
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 3: Finished eating (iteration 7)
Philosopher 3: Thinking (iteration 8)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 0: Finished eating (iteration 5)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 0: Thinking (iteration 6)
Philosopher 4: Finished eating (iteration 6)
Philosopher 4: Thinking (iteration 7)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 2: Finished eating (iteration 7)
Philosopher 2: Thinking (iteration 8)
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 3: Finished eating (iteration 8)
Philosopher 3: Thinking (iteration 9)
Philosopher 1: Finished eating (iteration 6)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 1: Thinking (iteration 7)
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 2: Finished eating (iteration 8)
Philosopher 2: Thinking (iteration 9)
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 0: Finished eating (iteration 6)
Philosopher 0: Thinking (iteration 7)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 3: Finished eating (iteration 9)
Philosopher 3: Thinking (iteration 10)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 1: Finished eating (iteration 7)
Philosopher 1: Thinking (iteration 8)
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 4: Finished eating (iteration 7)
Philosopher 4: Thinking (iteration 8)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 2: Finished eating (iteration 9)
Philosopher 2: Thinking (iteration 10)
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 3: Picked up chopstick 3 (first)
Philosopher 3: Picked up chopstick 4 (second) - EATING
Philosopher 1: Finished eating (iteration 8)
Philosopher 1: Thinking (iteration 9)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 3: Finished eating (iteration 10)
Philosopher 3: All done!
Philosopher 2: Picked up chopstick 2 (first)
Philosopher 2: Picked up chopstick 3 (second) - EATING
Philosopher 0: Finished eating (iteration 7)
Philosopher 0: Thinking (iteration 8)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 2: Finished eating (iteration 10)
Philosopher 2: All done!
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 1: Finished eating (iteration 9)
Philosopher 1: Thinking (iteration 10)
Philosopher 4: Finished eating (iteration 8)
Philosopher 4: Thinking (iteration 9)
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 0: Finished eating (iteration 8)
Philosopher 0: Thinking (iteration 9)
Philosopher 1: Picked up chopstick 1 (first)
Philosopher 1: Picked up chopstick 2 (second) - EATING
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 4: Finished eating (iteration 9)
Philosopher 4: Thinking (iteration 10)
Philosopher 1: Finished eating (iteration 10)
Philosopher 1: All done!
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 0: Finished eating (iteration 9)
Philosopher 4: Picked up chopstick 0 (first)
Philosopher 4: Picked up chopstick 4 (second) - EATING
Philosopher 0: Thinking (iteration 10)
Philosopher 4: Finished eating (iteration 10)
Philosopher 4: All done!
Philosopher 0: Picked up chopstick 0 (first)
Philosopher 0: Picked up chopstick 1 (second) - EATING
Philosopher 0: Finished eating (iteration 10)
Philosopher 0: All done!
All philosophers finished without deadlock!
dcuev@iFHOX:~/cosc315/Lab6$ |

```

Task 5:
Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>

#define NUM_PHILOSOPHERS 5
#define NUM_ITERATIONS 3

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;
    int meals = 0;

    while (meals < NUM_ITERATIONS) {

        printf("Philosopher %d: Thinking\n", id);
        fflush(stdout);
        usleep((rand() % 100 + 50) * 1000);

        pthread_mutex_lock(&chopsticks[left]);
        printf("Philosopher %d: Picked up LEFT chopstick %d\n", id, left);
        fflush(stdout);

        struct timespec timeout;
        clock_gettime(CLOCK_REALTIME, &timeout);
        timeout.tv_sec += 1;

        int result = pthread_mutex_timedlock(&chopsticks[right], &timeout);

        if (result == ETIMEDOUT) {

            printf("Philosopher %d: Timeout! Releasing chopstick %d and retrying...\n", id, left);
            fflush(stdout);
            pthread_mutex_unlock(&chopsticks[left]);

            usleep((rand() % 200 + 100) * 1000);
            continue;
        }

        printf("Philosopher %d: Picked up RIGHT chopstick %d - EATING (meal %d)\n",
            id, right, meals + 1);
        fflush(stdout);

        usleep((rand() % 100 + 50) * 1000);

        pthread_mutex_unlock(&chopsticks[right]);
        pthread_mutex_unlock(&chopsticks[left]);
        printf("Philosopher %d: Finished meal %d\n", id, meals + 1);
        fflush(stdout);

        meals++;
    }

    printf("Philosopher %d: All done!\n", id);
    return NULL;
}

int main() {
    srand(time(NULL));
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_init(&chopsticks[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_join(threads[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_destroy(&chopsticks[i]);

    printf("All philosophers finished with timeout-based recovery!\n");
    return 0;
}

```

Output:


```

dcuev@iFHOX:~/csc315/lab6$ ./philosophers_timeout
Philosopher 0: Thinking
Philosopher 1: Thinking
Philosopher 3: Thinking
Philosopher 4: Thinking
Philosopher 2: Thinking
Philosopher 1: Picked up LEFT chopstick 1
Philosopher 1: Picked up RIGHT chopstick 2 - EATING (meal 1)
Philosopher 3: Picked up LEFT chopstick 3
Philosopher 3: Picked up RIGHT chopstick 4 - EATING (meal 1)
Philosopher 0: Picked up LEFT chopstick 0
Philosopher 3: Finished meal 1
Philosopher 3: Thinking
Philosopher 4: Picked up LEFT chopstick 4
Philosopher 1: Finished meal 1
Philosopher 1: Thinking
Philosopher 2: Picked up LEFT chopstick 2
Philosopher 2: Picked up RIGHT chopstick 3 - EATING (meal 1)
Philosopher 0: Picked up RIGHT chopstick 1 - EATING (meal 1)
Philosopher 2: Finished meal 1
Philosopher 2: Thinking
Philosopher 0: Finished meal 1
Philosopher 0: Thinking
Philosopher 1: Picked up LEFT chopstick 1
Philosopher 1: Picked up RIGHT chopstick 2 - EATING (meal 2)
Philosopher 4: Picked up RIGHT chopstick 0 - EATING (meal 1)
Philosopher 3: Picked up LEFT chopstick 3
Philosopher 4: Finished meal 1
Philosopher 4: Thinking
Philosopher 3: Picked up RIGHT chopstick 4 - EATING (meal 2)
Philosopher 3: Finished meal 2
Philosopher 3: Thinking
Philosopher 0: Picked up LEFT chopstick 0
Philosopher 1: Finished meal 2
Philosopher 1: Thinking
Philosopher 0: Picked up RIGHT chopstick 1 - EATING (meal 2)
Philosopher 2: Picked up LEFT chopstick 2
Philosopher 2: Picked up RIGHT chopstick 3 - EATING (meal 2)
Philosopher 4: Picked up LEFT chopstick 4
Philosopher 0: Finished meal 2
Philosopher 0: Thinking
Philosopher 1: Picked up LEFT chopstick 1
Philosopher 4: Picked up RIGHT chopstick 0 - EATING (meal 2)
Philosopher 2: Finished meal 2
Philosopher 2: Thinking
Philosopher 3: Picked up LEFT chopstick 3
Philosopher 1: Picked up RIGHT chopstick 2 - EATING (meal 3)
Philosopher 4: Finished meal 2
Philosopher 4: Thinking
Philosopher 3: Picked up RIGHT chopstick 4 - EATING (meal 3)
Philosopher 0: Picked up LEFT chopstick 0
Philosopher 1: Finished meal 3
Philosopher 1: All done!
Philosopher 2: Picked up LEFT chopstick 2
Philosopher 0: Picked up RIGHT chopstick 1 - EATING (meal 3)
Philosopher 3: Finished meal 3
Philosopher 3: All done!
Philosopher 4: Picked up LEFT chopstick 4
Philosopher 2: Picked up RIGHT chopstick 3 - EATING (meal 3)
Philosopher 0: Finished meal 3
Philosopher 0: All done!
Philosopher 4: Picked up RIGHT chopstick 0 - EATING (meal 3)
Philosopher 2: Finished meal 3
Philosopher 2: All done!
Philosopher 4: Finished meal 3
Philosopher 4: All done!
All philosophers finished with timeout-based recovery!
dcuev@iFHOX:~/csc315/lab6$ |

```

Task 6:

Conceptual Questions

Q1: In the original Dining Philosophers problem, identify which of the four deadlock conditions are present. Explain each.

A1: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait takes place in the Dining Philosophers problem.

Mutual Exclusion - Each chopstick can only be held by one philosopher at a time, they are not sharable resources.

Hold and Wait - Each philosopher picks up their left chopstick and holds it while waiting to get the right chopstick. They hold one resource while waiting for the other.

No Preemption - Chopsticks are assigned to each other and they cannot be taken from forcefully, unless the philosopher releases it voluntarily.

Circular Wait: Philosopher 0 waits for Philosopher 1, who waits for Philosopher 2, and so on until the last Philosopher waits for Philosopher 0.

Q2: The resource ordering solution breaks which deadlock condition? Explain why.

A2: Ordering the resources is what eliminates Circular Wait condition. Philosophers pick up the chopstick with a low number first, it creates a very strict sequence. At least one philosopher's first resource is also another's second.

Q3: What are the advantages and disadvantages of the timeout approach compared to resource ordering?

A3:

Timeout Advantages	Timeout Disadvantages	Resource ordering advantages	Resources ordering disadvantages
Works without knowing the full resource graph in advance. More flexible, it can be applied to any scenario. Useful when resources have complex ordering requirements	Doesn't prevent deadlock, only detects and recovers. The timeout duration has to be taken of carefully.	It prevents deadlock completely with zero overhead at runtime.	It needs to know all resources and their ordering. Difficult to apply when resources are dynamically created.

Q4: Could starvation occur in the resource-ordered solution? Why or why not?

A4: Yes, any philosopher could be delayed if the other philosophers who want the same chopsticks are always scheduled ahead of it. The solution doesn't guarantee starvation, it only helps for deadlock.